

CodeML: A Machine Learning-Assisted User Interface for Code Identification and Labeling

Francine Chen
Toyota Research Institute
Los Altos, CA, USA
francine.chen@tri.global

Kate Glazko[†]
Toyota Research Institute
Los Altos, CA, USA
kateglazko@gmail.com

Matthew K. Hong
Toyota Research Institute
Los Altos, CA, USA
matt.hong@tri.global

Emily Sarah Sumner
Toyota Research Institute
Los Altos, CA, USA
emily.sumner@tri.global

Laurent Denoue^{*}
Toyota Research Institute
Los Altos, CA, USA
ldenoue@gmail.com

Yan-Ying Chen
Toyota Research Institute
Los Altos, CA, USA
yan-ying.chen@tri.global

Matt Klenk
Toyota Research Institute
Los Altos, CA, USA
matt.klenk@tri.global

ABSTRACT

Labeling short, unstructured texts is generally performed by sequentially identifying *codes* and assigning them to segments of text based on viewing a small sample of data. In this greedy approach, coders risk overlooking important code ideas and must perform the tedious task of iteratively revising the initial code set, and sometimes response code assignments, as new themes emerge. To address this, we propose CodeML, a machine learning-assisted (ML) coding interface that identifies multiple ideas in a response, which are displayed to support interactive data exploration, code identification, and refinement of snippet code assignments. By surfacing themes and snippets early, coders can consider a broader range of potential codes to reduce chances of omitting codes that surface later. A comparative study against search-style coding shows the potential for CodeML to facilitate initial exploration and discovery of finer-grained code sets while not adding significant cognitive load to organize codes and underlying text snippets.

KEYWORDS

qualitative coding, interactive machine learning, text annotation

ACM Reference Format:

Francine Chen, Matthew K. Hong, Laurent Denoue, Kate Glazko, Emily Sarah Sumner, Yan-Ying Chen, and Matt Klenk. 2023. CodeML: A Machine Learning-Assisted User Interface for Code Identification and Labeling. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3544549.3585587>

^{*} Author is currently at Applbit

[†] Author is currently an Independent Researcher

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '23, April 23–28, 2023, Hamburg, Germany

© 2023 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9422-2/23/04.

<https://doi.org/10.1145/3544549.3585587>

1 INTRODUCTION

Short phrases in free text often need to be labeled or *coded*¹ for use by a variety of disciplines to meet different labeling goals. For example, the coded data may be used for human-led analysis of free-text responses, or it may be used to train or adapt machine learning models on new tasks or new domains. For some applications, identifying a few prominent themes as codes, possibly along with a few examples, is adequate. But other applications, e.g., for human behavior studies or for labeled training data, require a dataset be fully labeled. This enables a deeper, more complete analysis of the responses and allows for less popular, but possibly important, themes to be surfaced. This second type of application with complete analysis of text responses on a smaller size dataset is the target of our work.

Muller et al.[13] explored how human experts determine ground truth, or a set of codes. When there is no prior knowledge of the desired codes, the ground truth is primarily derived from examining the data. This task poses a significant cognitive burden on the coder. The sample of free responses in Figure 1 (Left) illustrate some of the issues faced when coding free-response text, including multiple codes per response, multiple surface forms with the same meaning, and redundancy.

In a common, greedy method for coding text, a coder reviews a subset of responses to propose a few initial codes, and then begins assigning codes to responses, modifying the set of codes as they work through a list of responses. The coder often uses search, which is offered by many coding tools such as MAXQDA [12]; this requires hypothesizing the phrases to associate with a code. As coders examine more data, they may add codes or decide that a code was not popular enough to keep, refining the *code set* by adding or removing codes. The greedy approach of defining codes based on examining a subset of codes can result in overlooked themes present in the data. This can happen for a set of interrelated reasons: (1) a theme may have rarely occurred in the first sample of

¹In this paper we will use the term ‘coding’ to refer to the labeling of short phrases, which is also referred to as ‘labeling’ or ‘annotation’ in some fields.

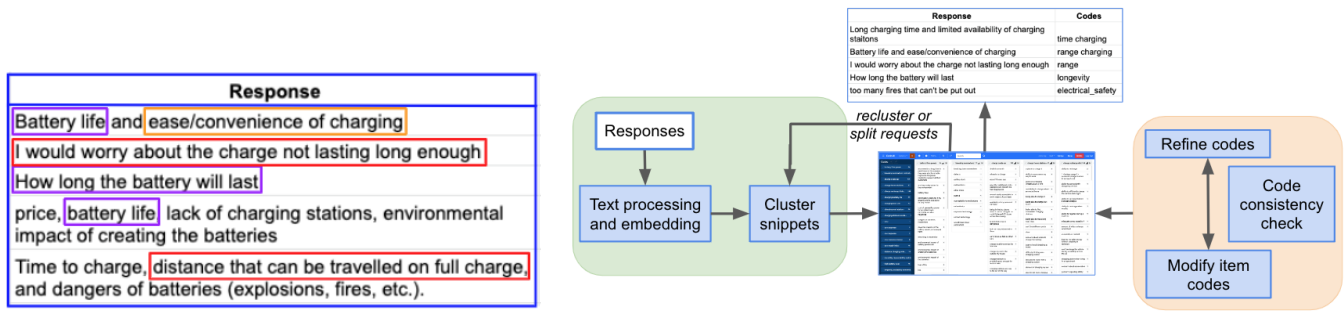


Figure 1: (Left) Sample of free text responses examined during coding. Coding the responses requires accounting for: multiple codes may be needed for one response (e.g., the first response should be labeled with two codes), multiple ways to refer to the same concept (e.g., the purple items convey similar concepts and the red items convey similar concepts), and redundant items (e.g., "battery life"). (The sample of responses were to a question about barriers to buying or leasing a battery electric vehicle.) (Right) Overview of the CodeML approach. The interface integrates input from the machine learning components (green) with humans (orange) who use their knowledge to refine the ML-proposed codes.

responses, (2) some of the phrases, or *snippets*, in a response may be overlooked in phrases containing multiple ideas. (3) a theme may have many varied expressions, some of which may not be quickly discerned to have the same meaning.

We are developing a system, **CodeML**, to provide ML-based assistance to supporting coding (see Figure 1 (Right)). CodeML provides an interactive interface between humans, who bring their knowledge about the task to decide what the codes should be, and machine learning, for suggesting ways to group snippets and split clusters. To reduce bias when performing code identification, CodeML initially proposes themes to a coder to help identify the primary themes in a response set. To support the code identification and code labeling tasks, CodeML also helps a coder to interactively define codes and modify the snippets assigned to a code. In contrast to common machine learning-based systems where human input is secondary, CodeML requires human input to guide the selection and definition of the codes.

Our work contributes 1) a new, non-greedy approach, CodeML, in an ML-assisted interface for identifying a code set for labeling short, free-response text that pre-processes the responses into *snippets* and then proposes groups of similar concepts as codes to reduce the possibility of overlooking themes in the responses, and 2) an evaluation against a greedy, search-style approach in a popular commercial product (i.e., ATLAS.ti), demonstrating that CodeML supported identifying codes at a finer level for a deeper understanding of the dataset. Many user participants thought that CodeML would be preferable when a coding required more than identifying a few high-level themes and examples.

2 RELATED WORK

Commercially available tools augment traditional spreadsheet style coding by offering the ability to search for keywords, such as from the results of a boolean search or a word cloud, and to assign code labels to text segments that match, e.g., MAXQDA [12], NVivo[14], and ATLAS.ti[1]. There are known benefits to search-style auto-coding, but its effectiveness in supporting early code identification remains an open question. For instance, Marathe[11] and Rietz[15] demonstrated the value of advanced query-style coding, but such an approach requires upfront knowledge of candidate codes after

having done a first-pass code identification as well as practical experience using boolean search operators and modifiers.

Tools that incorporate ML into the coding process exist as products as well as published research prototypes. Business intelligence tools such as Microsoft Power BI[4], Canva AI[5], and Thematic[18] use a suite of natural language processing techniques to provide summary statistics for an unstructured document corpus, which typically include the use of word co-occurrences, sentiment analysis, and named entity recognition, among others. While these tools provide a quantitative treatment to unstructured and qualitative data, coders have little control over how the codes are generated and without much context about the raw data.

Recently, a number of tools include a complete package supporting the exploration and management of coded data. For example, Amazon SageMaker [2] classifies text into categories that have been predefined. Displayr [7] offers an interface for defining new categories one at a time, which can then be used for labeling unlabeled text. However, it does not indicate coverage of the defined categories. NVivo [14] provides an ML-based pattern coding feature which, after having initial codes generated by the users, allows users to specify which machine-suggested keywords to use in detecting phrases that fall under the code set. More recently, Rietz et al.[15] developed Cody, a system that allows users to define advanced query-style code rules in combination with supervised machine learning. While all of these tools facilitate semi-automated coding of large datasets, they require a predefined label set, and give users little to no control over how codes are applied.

Prior academic work has explored coding support with interactive topic modeling systems that prompt users with a tool to revise the topic model by creating, splitting, and merging topics. ConVisIT provides a visualization of a set of topics extracted from an online forum to support exploration of post and comment threads. Users can interactively revise the topic model by splitting and merging topic nodes, triggering retraining of the model[9]. Bakharia et al. [3] evaluated two different topic modeling algorithms with domain experts who relied on their knowledge of topics in a dataset to specify create, merge or split rules and revise generated themes. While these systems employ ML to support initial code discovery and identification, the resulting codes are coarse-grained because entire sentences and paragraphs are clustered. Moreover, an overview of

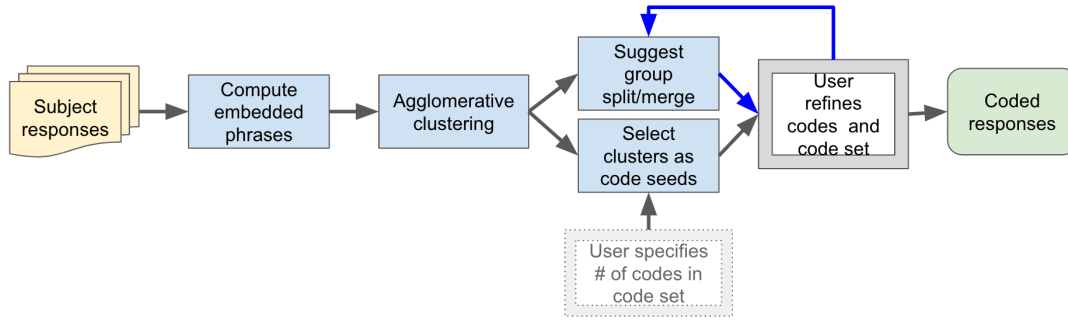


Figure 2: The machine learning processing for CodeML, shown in blue, suggests ways to group snippets and split groups of snippets. The user considers these suggestions as they refine codes and code sets.

free-text responses is not possible; instead, the systems prioritize visualization and user interaction with topics alone. In contrast, CodeML breaks down raw responses into discrete ideas to ensure that each of the ideas is coded. It visualizes groupings of similar text snippets in an interface to support interactive refinement of the groups, labels, or codes of each group.

3 APPROACH: ML ASSISTANCE FOR CODING

The approach in CodeML for creating a code set is to support identification of the primary themes expressed in a dataset and facilitate intra-user review of the snippets (i.e., response phrases) assigned the same code. Figure 1 (Right) shows an overview of the CodeML AI-assisted interface. The responses to be coded are given to the text analysis component (green) with the resulting set of candidate codes and their assigned snippets displayed to a user (center). The user (right) can then refine the codes and assigned snippets, making use of machine-learning-based clustering and similarity computations. Once the user is satisfied with the codes and assignment of snippets to codes, the codes for each response can be output in the desired format (top). For example, by code and associated snippet (not shown in the figure), or by original free-text response and codes (as shown in the figure).

The task of defining a code set typically requires a user to first scan text responses to identify variations of portions of the responses that correspond to a code. CodeML is designed for responses such as those in Figure 1 (left), which are lists of one or more items. Rather than showing users full text responses, CodeML splits each response into snippets, with each snippet capturing one concept, as illustrated by the text in each colored box in Figure 1 (left). The CodeML system uses machine learning to suggest groupings of snippets (see Appendix B). This is shown in Figure 2, where the machine learning (ML) module (on the left) segments each input response into one or more text snippets (see Appendix B.1 for implementation details). Because each snippet corresponds to only one code, machine learning can use hard clustering to group text snippets that are semantically similar. As a result, the user need not explicitly code each response or text snippet individually. The snippet groups are then given to the CodeML interface for display to the user. The clusters are displayed as candidate codes, or concepts, for the user to consider. Each cluster ideally represents a theme that is expressed in its snippets. Thus themes that may be overlooked

with a greedy approach to defining codes because the codes cover many responses can now be surfaced. If the initial clusters are too coarse or too fine, the user can modify the number of clusters (i.e., codes) to adjust the granularity and view the results until the user is satisfied with the initial set of clusters.

The user then refines the cluster labels (suggested codes) and snippets assigned to each cluster, as well as the number of clusters as they view the refined clusters. Snippets that are out of place can be dragged to another cluster, either separately or as a group (copy and paste is also an option).

During the revision of the proposed snippet groupings, upon the user requesting that a cluster be split, machine learning can provide support by suggesting how to split a group of snippets, which the user can inspect and reject if desired. If a group appears to contain more than one idea, the user can ask the ML module to cluster the snippets in the group into two sub-clusters, splitting the group into two clusters. The original cluster is replaced by the new sub-clusters in the interface, where each sub-cluster is represented as a new cluster among the existing clusters. Sub-themes that occur in the cluster or group can then be surfaced. The user can repeatedly ask to have a group split.

The user can also merge clusters into one and merge snippets from different clusters into an existing or new cluster. Thus, the number of clusters is determined by the user’s perspective of the number of codes they think is needed to capture the concepts of interest for the dataset. CodeML provides initial code names by identifying frequent terms in the snippets assigned to a cluster. The users are expected to edit the computed initial code name assigned to each cluster which are displayed in the header of each cluster. By viewing the proposed groups of semantically similarity snippets, the user may uncover unexpected codes that occur in only moderate numbers that could be easily overlooked. The user can check the new groups for consistency and possibly refine each.

4 CODEML INTERFACE OVERVIEW

CodeML provides a method for viewing groups of similar snippets to help in identification of a code set and also to simultaneously propose snippets that belong to each code in a code set. The set of codes often depends on the distribution of text responses, and so to get a sense of the granularity of codes needed, a user can quickly browse the set of groups of similar answers. In addition,

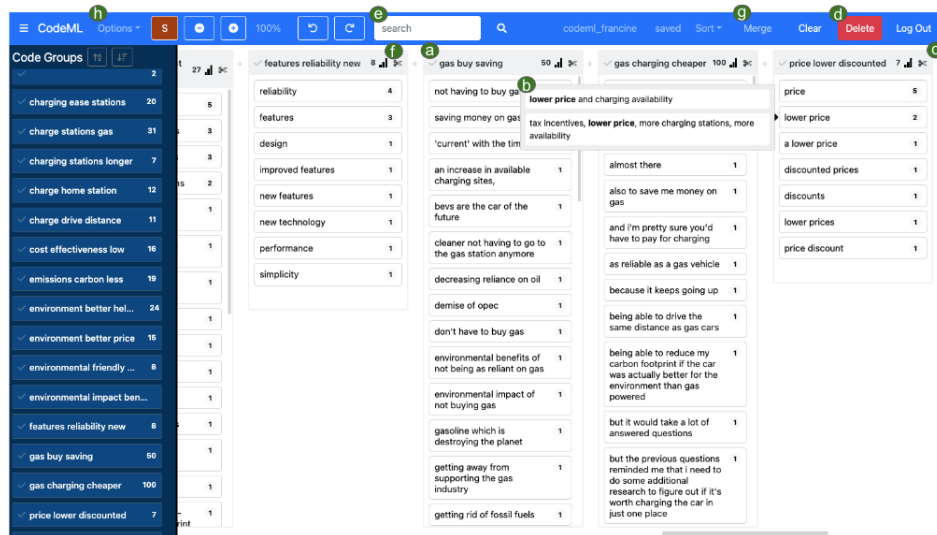


Figure 3: The interface for CodeML before a user has begun coding. The columns in the white main panel correspond to the computed, proposed groups of snippets which a user is to review and modify. The column headers (a) are labeled with the most frequent terms in each group as suggested code names. The dark blue Code Groups correspond to the column headers and can be clicked to bring any column into view or moved to revise the column order. By hovering over a snippet, the tooltip allows a user to quickly view a snippet in the context of the full response(s) (b). Other actions include (c) create new code, (d) delete snippets or a code group, (e) search, (f) split a cluster, (g) merge two clusters and (h) recluster the snippets or export the coded responses under 'Options'.

combining codes at a fine granularity can be performed in one simple operation to merge groups in CodeML's interface. Because the clustering will contain "errors", the user can use the interface to refine the initial set of clusters.

The main (white) panel initially displays the code groups, each labeled with the computed summary of frequent terms (Figure 3). The number of instances of each snippet is shown next to the snippet text. The display of snippets assigned to the same code allows a user to view many text snippets simultaneously and to check for consistency in the defined labels and codings. The user also can refine which text snippets are assigned to a code, as well as the code set. The system also helps to guide the user when deciding which text snippets labeled with a code to split.

On the left side of the interface, the list of "codes" are displayed in dark blue. A code can be clicked to display in the main panel the group of snippets corresponding to the selected code. Core features and capabilities of the interface (shown in Figure 3) include:

Code name suggestions Each column (a) in the main panel is headed with the name of a code and underneath the different responses as well as a count of those responses that are assigned that code. The initial name is automatically assigned, but can be revised by the user.

View full response The context of the snippets can be seen by hovering over a snippet to see the context from which the snippet was extracted (b).

Adding, moving, and deleting snippets New code groups can be created by clicking the '+' (c) between each cluster. The snippets can be moved from one code to another or to a new code group. In addition, multiple code groups can be merged into one. Snippets and code groups can also be deleted if needed (d).

Search Search (e) for a character sequence or phrase is supported. The results are shown as highlighted snippets in yellow which float

to the top of each code group. The codes in the Code Group list in dark blue on the left which contain a search result are also highlighted in yellow. These codes can be clicked to scroll the interface to display the corresponding code group.

Suggested splits A user can ask for a suggested split of the snippets labeled with a code by clicking on the scissors icon in the cluster heading (f). The ML component is given the snippet IDs, which it clusters into two groups that replace the original code group in the CodeML display.

Merge A user can merge two clusters by selecting each cluster and then clicking the merge button (g).

Cluster number Under the Options button (h), the user can specify the desired number of clusters.

5 EVALUATION

5.1 Study procedure

To investigate whether and how CodeML helped participants explore and identify an initial set of codes in an unstructured text corpus, we recruited 12 participants (5 female, 7 male; age $M=38.2$ and $SD=11.4$) to participate in an online within-subjects study in which we compared CodeML against a simple search style² coding feature in ATLAS.ti. Our inclusion criteria included people who are fluent in English, over 18 years of age, completed at least 1 coding task in the past year, and have experience coding, annotating or labeling unstructured data (See Appendix C for participant demographics). Our study also included a qualitative evaluation of CodeML to better understand people's perspectives after experiencing the two different coding techniques.

²While Marathe[11] and Rietz[15] demonstrated the value of advanced query-style coding, such an approach requires upfront knowledge of candidate codes after having done a first-pass code identification as well as practical experience using boolean search operators and modifiers.

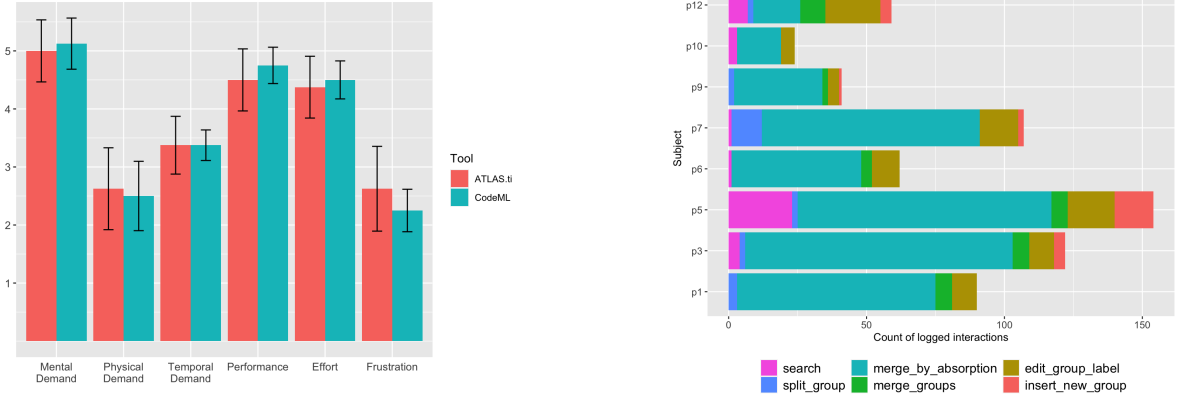


Figure 4: (Left) Likert ratings ($n=8$) across six subscales of the NASA-TLX questionnaire. On average, participants reported experiencing slightly more mental demand, effort, and sense of accomplishment (i.e., performance) with CodeML, but there was no statistically significant difference in perceived cognitive load between coding in CodeML and ATLAS.ti across all six dimensions of the NASA-TLX scale. (Right) Frequency distribution of logged interactions across participants who completed the coding task in CodeML.

Participants were asked to identify and label codes using two tools, CodeML and ATLAS.ti, each on one of two datasets (described in Appendix A). The order of tool presentation was counterbalanced such that half of the participants used CodeML first and the other half started with ATLAS.ti. Prior to beginning the task, participants reported a moderate familiarity ($M=2.92$, $SD=1.37$) with the topic covered in both datasets (e.g., pros and cons of electric vehicles). Following a 10-minute training phase, participants were given 30 minutes to complete a code identification and labeling task using each tool. Each participant was instructed to spend the first three minutes reviewing the data, determine the number of clusters to work with, and verbalize their thoughts (i.e., think-aloud) as they began coding. Upon completing each task, participants were asked to fill out the NASA-TLX questionnaire[8].

A 3rd party moderator facilitated the study and conducted a 30-minute interview after participants completed both tasks. Interview questions focused on participant experiences related to their ability to gain high- and low-level understanding of the data and perspectives about using ML to identify and label codes. In total, the study lasted 150 minutes and participants were compensated with 275 USD in gift card money.

For quantitative analysis, we performed a series of non-parametric repeated measures tests (Wilcoxon signed-rank test) to determine statistically significant difference in reported cognitive load levels (NASA-TLX) after using CodeML and ATLAS.ti. Additionally, we summarized CodeML usage log data using descriptive statistics (see Appendix C.2 for results of statistical analyses). For qualitative data, following [6], the second author and two research assistants independently and openly coded 12 participants' responses to interview questions and think-alouds, taking note of emerging themes. The three researchers iteratively discussed and refined these themes and resulting code sets until no new themes emerged.

In reporting the study results, we interleave quantitative results from the eight participants who completed the task³ with qualitative results from all participants who shared their experience and perspectives after having used both tools.

5.2 Results: initial data exploration and code identification

When asked to reflect on their ability to gauge themes using CodeML, more than half of our participants (p1-3, p5, p7, p10, p12) agreed that CodeML facilitated both high- and low-level overview of the data. For instance, p1 stated: “It’s always good to have the option to have a big picture, and then also a more detailed picture, and I feel like CodeML gives you both of those.” While less enthusiastic about the quality of the initially suggested code groups, p7 still appreciated the ability to start from the machine-suggested themes. “I don’t think it really like cut that much work out of it... But overall, it was pretty easy to like go from there to something more coherent.”

Our analysis of resultant codes shows the breadth and depth of the unique themes generated with CodeML when compared to those of ATLAS.ti. Table 1 shows unique codes (in red) captured by all participants who completed their tasks in the given time. In the barriers dataset, for example, participants using ATLAS.ti only identified *availability of charging stations* for *charging*, whereas those using CodeML identified an additional subset of *charging—charging time*. In the motivators dataset, CodeML participants identified fine-grained variants of *cost* or *cost savings* such as *tax incentives* and *discounts*. Moreover, with CodeML, participants identified a breadth of unique themes including *performance*, *battery replacement*, *unfamiliarity with technology*, *top speed*, *size* and *appearance*.

While highlighting CodeML’s ability to detect themes, some participants drew comparisons to their relatively poor experience with ATLAS.ti (p1, p5, p10). For example, p10 has likened the frustrating experience of having to look for the themes in a document to a

³We excluded four participants’ data (p2, p4, p8, p11) from the quantitative analysis because they either did not understand and complete the task, accessed from a foreign country or accidentally reinitialized the CodeML software, resulting in data loss or corruption.

Table 1: Codes captured in CodeML and ATLAS.ti. Within each dataset, codes unique to ones captured in either tool are color-coded in red. Numbers in parentheses indicate the count of same codes observed across participants.

Barriers		Motivators	
CodeML (p1,3,5,7,9)	distance/range (4), availability of charging stations (2), charging time (3) , safety (4), expensive energy (2), cost (2), price (2), environment (3), reliability, performance, battery replacement, unfamiliarity with tech, top speed, size, appearance	CodeML (p6,10,12)	environment friendly (2), charging (2), distance/range (2), features (2), tax incentives (2), discounts (3) , ease/benefits of charging (2), maintenance, convenience , gas charging
ATLAS.ti (p6,10,12)	distance/range (3), availability of charging stations (2), price (2), battery (2), safety (2), convenience (2), maintenance , cost, environment, expensive, reliability	ATLAS.ti (p1,3,5,7,9)	environment friendly (5), charging (3), distance/range (3), avoid buying gas (3) , cost (2), cost savings (3), reliability (2), safety (2), quietness (2) , features

“fishing expedition”. P5 commented on why search alone was not sufficient enough for them to quickly determine the underlying themes in the raw text data in plain view. “I might not immediately think that, like price and cost and expensive, were all kind of getting at the same thing, so I have to already be familiar with these synonyms.” In addition, while ATLAS.ti’s autocoding search interface provides suggestions for synonyms, participants who tried using it became frustrated because most of its suggestions had little contextual relevance to the dataset.

Despite the benefits of having both coarse and fine-grained themes from the start, providing fine-grained themes early on came at the cost of added cognitive load to process the data because segmenting responses into single word or phrase level snippets created more discrete *units of analysis* [16]) to peruse. Our quantitative analysis of perceived cognitive load also supported this view, although it was not statistically significantly different from coding in ATLAS.ti (Figure 4 (Left)). While it was perceivable more work, P1 still liked CodeML’s pre-generated labels because it gave him the ability to approach coding in a subtractive manner. He stated, “I didn’t mind [reading text snippets] because it’s like once, you know, ‘Okay, this category is availability.’ You just kind of knock out everything that’s not availability.”

Overall, our findings suggest CodeML’s potential to facilitate high-level overview of all themes presented in a text corpus before proceeding to a more targeted coding approach, which we discuss below.

5.3 Results: organizing codes

In organizing the data in CodeML, all participants mentioned reviewing each cluster by weeding out irrelevant text snippets until the cluster only contained semantically coherent ones. While people were excited that CodeML “took the first step” to organize the data, there were mixed sentiments about the work required to complete the coding. While acknowledging that CodeML forced them to “be a lot closer to the data” (p1), many claimed that the number of required interactions (e.g., drag and drop) and additional verification work to organize the data to their satisfaction was overwhelming. In contrast, participants rarely reviewed their codes and associated full-text responses in ATLAS.ti. Participants attributed this to the feeling of having “more control” over their codes, as p7 pointed out. Comparing with the imperfect clusters in CodeML, p4 commented that the search-style coding approach made him feel more in control. “because I started out filtering. I’m creating my list. I know I’m gonna go with it, and I can move along at a better pace.”

For some people, the presence of multiple granular-level themes in CodeML prompted them to rely less on search ($M=4.88$, $SD=7.70$).

When asked about their infrequent use of search, p1 remarked, “It was just about me kind of re-categorizing some of the stuff, but I never felt like I needed to search for anything”. P9, who also did not use search, stated that he would transition to a search-style coding approach if he had more time and completed his first pass in CodeML. Indeed, the participants (p3,p5,p10,p12) who quickly organized the clusters in CodeML moved on to using search as a way to further expedite the removal of irrelevant snippets or create new clusters with the returned search results.

While participants organized their codes by moving snippets into groups and using search queries, they rarely merged groups ($M=4.13$, $SD=3.23$) or split ($M=2.75$, $SD=3.49$) groups. People mentioned that splitting was not necessary as they started with too many clusters ($M=14.38$, $SD=6.05$) and would only consider using both split and merge features more often if they needed to code larger datasets. P3, who only used the split feature twice, said she would use it more once she became familiar with the feature, remarking “the scissor is helpful in the sense that if you might say, ‘Hmm. There are probably some sub-themes in here that are getting buried.’”

In summary, our results suggest that, in addition to aiding code discovery, CodeML facilitates iterative refinement of codes via an interactive interface that supports users in controlling the arrangement of clusters and text snippets. We also observed gaps in relying on search alone in ATLAS.ti. Due to their perceived feeling of ‘being in control,’ people seldom reviewed the coded responses, thereby increasing their chances of overlooking hidden themes in the data.

6 DISCUSSION, CONCLUSIONS AND FUTURE WORK

We proposed a novel application of ML to support early code identification and labeling, and a comparative evaluation against a popular commercial product. In contrast to the traditional method of coders proposing words and phrases to define a code, CodeML proposes groups of similar phrases in an interface that supports exploration and refinement of the groups. In a user study, we explored the utility of preprocessing text into snippets, which are each assigned a code.

Both our quantitative and qualitative analyses demonstrate potential opportunities and value in using ML to support initial code discovery and identification. Presenting both coarse and fine-grained themes in the very beginning offered participants the unique advantage of quickly discovering patterns in data that they would not have anticipated without perusing the data in a line-by-line fashion. Our study results further demonstrate that, with similar

amount of cognitive effort (when compared against a commercial product), participants are able to generate more unique codes with CodeML than ATLAS.ti in both datasets. Our participant's use of search to augment their data "cleaning" process after organizing the clusters to further filter irrelevant codes suggests a promising future complementary approach that considers both interactive clustering and query-based code rule generation in ways that facilitate subtractive and additive coding. While this finding contradicts Marathe et al.'s [11] recommended practice of semi-automating coding with search style code rule generation upon completing first-pass coding, our results provide new insights that highlight the unique benefits of involving machine learning in this early phase of coding.

Based on our findings, we also note that participants may find the abrupt transition to a new coding style challenging when involving ML early in the process—thus calling for an onboarding experience to help first-time users engage with key ML concepts and functionality to ensure a smooth transition from their prior coding style [17].

As with many HCI studies, there are limitations to the generalizability of our study results. First, the ecological validity of the task and dataset may not match the real-world practice of our participants, e.g., participants coded datasets that were not their own, and the short, list-type text responses in our dataset may be different from the type of data they commonly code. Moreover, due to the nature of the within-subjects study design, we limited each task to 30 minutes, which may limit the observed and self-reported experiences to first impressions of the tool. Despite these limitations, we are able to demonstrate CodeML's potential and value in facilitating quick discovery of fine-grained codes, as in the examples of *charging vs. charging time* or *cost savings vs. tax incentives* in Table 1, which were not readily surfaced using a traditional search-style approach alone. Finally, we did not account for individual differences in coding ability due to the difficulty in recruiting participants that meet our inclusion criteria and lack of standardized means for measuring such a highly specialized skill.

To fully support a system for coding with this approach, a number of directions would be useful. In the future, we would like to provide an easier way for identifying codes which are similar to a selected set of snippets or code for use in merging. Merging suggestions could be identified by computing the distance between the average embedding of each cluster pair and visually presented, perhaps with a layout based on distances between codes. Because some subjects would have liked the initial grouping to have fewer errors, we also would like to explore the use of another encoder, such as RoBERTa, which had a *sentence* encoding performance of 92.4% vs. the BERT encoder's 90.0% on the GLUE sentence similarity task (STS) [10]. As improvements are made to models for embedding short pieces of text for computing the similarity between snippets, new models could be used to further reduce the load on users to correct the initial groups. To further improve the clustering, online machine learning could suggest changes to clusters as the snippets assigned to a code are manually refined. While we used a simple approach for computing snippets from list-style inputs, as we look to handle longer texts in the future, a parser could process each response to identify the implied context of each snippet. Finally, these future improvements must accompany appropriate human-centered measures to mitigate algorithmic bias

and human over-reliance in the context of ML-assisted qualitative coding.

While not a complete solution to the coding problem, our evaluation of CodeML illustrates the promise and complexities of integrating ML into the cognitively burdensome task of coding. To this end, a future larger-scale study after adding UX enhancements to the user interface—based on feedback we received in this study—will allow us to more fully realize the potential of CodeML.

REFERENCES

- [1] Atlas.ti. 2022. Auto-coding and Smart Coding in ATLAS.ti Web. <https://atlas.ti.com/research-hub/auto-coding-and-smart-coding-in-atlas-ti-web>
- [2] Amazon AWS. 2022. Amazon SageMaker Data Labeling Features. <https://aws.amazon.com/sagemaker/data-labeling/features>
- [3] Aneesh Bakharia, Peter Bruza, Jim Watters, Bhuva Narayan, and Laurianne Sitbon. 2016. Interactive topic modeling for aiding qualitative content analysis. In *Proceedings of the 2016 ACM on Conference on Human Information Interaction and Retrieval*. 213–222.
- [4] Microsoft Power BI. 2022. Use AI Insights in Power BI Desktop. <https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-ai-insights>
- [5] Canvs. 2022. Canvs MRX: Open Ends to Insights. <https://canvs.ai/canvs-mrx/>
- [6] Victoria Clarke, Virginia Braun, and Nikki Hayfield. 2015. Thematic analysis. *Qualitative psychology: A practical guide to research methods* 222, 2015 (2015), 248.
- [7] DisplayR. 2022. How to Semi-Automatically Code Text Data. <https://help.displayr.com/hc/en-us/articles/360003141436-How-to-Semi-Automatically-Code-Text-Data>
- [8] Sandra G Hart and Lowell E Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Advances in psychology*. Vol. 52. Elsevier, 139–183.
- [9] Enamul Hoque and Giuseppe Carenini. 2015. Convisit: Interactive topic modeling for exploring asynchronous online conversations. In *Proceedings of the 20th International Conference on Intelligent User Interfaces*. 169–180.
- [10] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).
- [11] Megh Marathe and Kentaro Toyama. 2018. Semi-automated coding for qualitative research: A user-centered inquiry and initial prototypes. In *Proceedings of the 2018 CHI conference on human factors in computing systems*. 1–12.
- [12] MAXQDA. 2022. Questionnaire Analysis with MAXQDA. <https://www.maxqda.com/questionnaire-analysis>
- [13] Michael Muller, Christine T Wolf, Josh Andres, Michael Desmond, Narendra Nath Joshi, Zahra Ashktorab, Aabhas Sharma, Kristina Brimijoin, Qian Pan, Evelyn Duesterwald, et al. 2021. Designing ground truth and the social life of labels. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [14] NVivo. 2022. About Automatic Coding Techniques. <https://help.nvivo.com/en-us/desktop/concepts/about-automatic-coding-techniques.htm>
- [15] Tim Rietz and Alexander Maedche. 2021. Cody: An AI-based system to semi-automate coding for qualitative research. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–14.
- [16] Margaret R Roller and Paul J Lavrakas. 2015. *Applied qualitative research design: A total quality framework approach*. Guilford Publications.
- [17] Brendan Strahm, Colin M Gray, and Mihaela Vorvoreanu. 2018. Generating mobile application onboarding insights through minimalist instruction. In *Proceedings of the 2018 Designing Interactive Systems Conference*. 361–372.
- [18] Thematic. 2022. Insights and Research. <https://getthematic.com/use-cases/insights-and-research/>